

S.U.R.E. (Aquaculture Welfare Integration): Autonomous Sturgeon Welfare Monitoring in Recirculating Aquaculture Systems via Edge-AI and RAG-Enhanced Deterministic Fusion

Batuhan Çıtak

Erdem Sabri Veli

Abstract

An implemented sturgeon welfare-monitoring system for recirculating aquaculture pairs a locally hosted LLM with a deterministic rule engine holding final, escalation-only authority over severity decisions, because a dissolved-oxygen crash can kill stock within hours and a silent model mistake is not tolerable. One might expect the architecture’s value to come from the rule engine correcting the model’s occasional wrong call. Measured against the system, offline, against synthetically generated sensor readings and labeled imagery, that is not what happens: half the evaluated scenarios produce output the model never parses, and the rule engine’s contribution is defaulting safe rather than adjudicating a disagreement – genuine correction occurs in only one case in eight. Even where the model’s final answer agreed with the rule engine, its reasoning fabricated a sensor value never present in the input in two of three such cases, invisible to output-only evaluation. This was tested against a self-documented, insufficient training adapter, making it a stronger stress test, not a weaker one. The same narrow-interface discipline held across four further layers: an agentic tool-routing negative result surviving an adversarial re-run, a retrieval threshold tuned for asymmetric cost, a drift-detection binning choice whose sensitivity crosses over rather than behaving monotonically, and a vision pipeline whose configured operating point runs at recall 0.782, not the headline academic-optimum recall of 0.719 usually quoted for it. The resulting rule is narrow and falsifiable: such a pipeline can be made safe against an unreliable model provided its validator reads nothing beyond one enumerable field and always defaults conservatively – a claim about interface design, not about having made the model trustworthy.

1 Introduction

Dissolved oxygen below roughly 6 mg/L can kill farmed sturgeon within hours, not days [Lindholm-Lehto, 2023, AgMRC, 2026]. A locally hosted large language model is genuinely useful in this setting – it can reason jointly over sensor readings and fish-behavior video, and explain its reasoning in plain language, in a way a fixed threshold table cannot – but an LLM that silently misses a safety-critical threshold is not a tool anyone should deploy unattended. S.U.R.E. is an implemented, real-time-capable welfare-monitoring architecture for sturgeon in a recirculating aquaculture system (RAS) that answers this tension with a specific architectural choice: a locally hosted LLM (AQUA-1B, with a LoRA domain adapter) proposes a severity assessment, but a deterministic rule engine has the final word, and can only ever raise the model’s proposed severity, never lower it. This paper reports what happened when that architecture was measured, not merely designed – across the decision layer itself and four other layers of the same system.

No physical water-quality sensor exists for this system, and none was built for this paper. That absence shapes what kind of contribution this is, and it is worth stating plainly rather than only as a caveat: without real sensor hardware, this work demonstrates the architectural feasibility of

running a multi-component pipeline – a locally hosted LLM, a computer-vision detector, a retrieval-augmented knowledge base, and a deterministic rule engine – together, and specifically tests whether that pipeline’s space and memory footprint is viable on realistic edge hardware. Framed this way, the paper is a feasibility and resource-management study of the architecture, run against synthetically generated sensor values and real, hand-labeled camera imagery captured from the system’s own physical setup, not a field validation of decision-making accuracy in a live setting. That is a legitimate and, we think, useful contribution in its own right, distinct from and prior to the question of whether the model’s decisions are accurate against a live water body.

Question. Does escalation-only deterministic authority over an LLM’s output survive contact with a genuinely unreliable local model, in a system that is actually running, rather than only in a design diagram?

The proposal this system was originally built around assumed a specific answer: the model proposes a severity judgment, and the rule engine’s value is in catching the cases where that judgment is a dangerous under-estimate. That is a clean story, and it is not the one this paper’s measurements support.

Question. When the architecture “works,” what does that actually mean, once measured – sophisticated correction of a nuanced misjudgment, or something structurally different?

Measured directly against all eight of the evaluation harness’s built-in scenarios, using the real, unmodified override logic and the system’s configured model, the dominant pathway is not correction. Half the time, the model’s output does not parse into anything usable at all, and the rule engine’s entire contribution is defaulting to a safe state rather than adjudicating a disagreement. Only one of eight scenarios shows the textbook pattern of the model under-calling a threshold and the rule engine escalating it. And in a finding this paper considers its most novel single result, two of the three scenarios where the model’s final answer agreed with the rule engine did so via reasoning that cited a sensor value never present in the input – fabrication that is invisible to any evaluation checking only the output label, which is how most guardrail and agentic-benchmark literature is scored. This was measured, deliberately, against the weakest plausible version of the model component: the configured adapter trained on eight handwritten examples the project’s own fine-tuning pipeline documents as insufficient, while a properly sized, 128-example alternative sat unused on disk. That the safety property held regardless is stronger evidence for the architecture’s design than a result obtained against a carefully tuned model would have been, and this paper states that directly rather than treating the undertrained adapter as a limitation to apologize for.

Question. Does the same narrow-enumerable-interface discipline generalize across architecturally distinct layers of one system?

Four more measurements bear on this question, each earning its place independently rather than restating the first. Agentic tool-routing was abandoned in favor of deterministic code specifically because an adversarial re-run – designed to break the negative result, not confirm it – only strengthened it. The vision pipeline’s own confidence-threshold choice turned out to run at a different, and more reassuring, point on its precision–recall curve than the number this field would otherwise headline. INT8 quantization, widely assumed to be where an edge deployment pays for its speed, turns out to lose *less* accuracy than the unquantized export formats already do. And a drift-detection binning choice that credit-scoring literature already documents as sensitive to convention turns out, on this system’s own confidence distribution, to invert the naive expectation at low severity before confirming it at high severity.

This paper does not claim to have solved LLM unreliability, and that claim would not survive its own evidence: the LLM component measured here is, on its own terms, frequently unreliable. What

the evidence supports is narrower and, we think, more useful to a practitioner building a similar system: a falsifiable design rule under which that unreliability becomes harmless rather than merely mitigated. Architect the pipeline assuming the model will often produce nothing usable, verify the fallback default is conservative, and never let the validator read anything from the model beyond the one enumerable field it is designed to certify – and the model’s failure modes, including the ones this paper newly documents, stop being the system’s failure mode.

That discipline is worth taking seriously precisely because this paper’s own system still lapsed once in exactly the way it warns against: during the audit behind this paper, the system’s own retrieval-augmented knowledge base was found to still be serving a stale accuracy figure in one file it had corrected everywhere else – a knowledge-base entry the correcting commit missed by 43 minutes and that has sat uncorrected since. That the same audit process caught it is the argument for the discipline, not a reason to distrust the rest of this paper’s numbers – it is disclosed here, and again in Section 8, as a live, currently unresolved issue rather than a historical anecdote.

Two further scope statements belong here, early, alongside the two above, because they bound what “measured” means throughout this paper. First, every result reported here is an offline, bench-level evaluation of the implemented system – against synthetically generated sensor readings, static labeled imagery, and scripted or synthetic scenarios run against the codebase – and not a field deployment: no experiment in this paper involved real physical sensors or an operating recirculating aquaculture facility, and only the system’s interface, not a live deployment, was ever demonstrated. Second, S.U.R.E. was built for the TEKNOFEST Agricultural Technologies competition, but this team did not enter that competition and did not pass its preliminary evaluation round. Both points are stated again, in full, in Section 8.

Takeaway. Malformed-output fail-safe defaulting, not sophisticated correction, is the dominant realized safety pathway (50% versus 12% of scenarios) – Section 6.1, Figure 1.

Takeaway. Fabricated sensor values survive inside outputs whose final status agrees with the rule engine, invisible to status-only evaluation – Section 6.1.

Takeaway. The agentic tool-routing negative result replicates under an adversarial re-run built to break it, not merely confirm it – Section 6.2.

Takeaway. The vision pipeline’s configured operating point governs recall at 0.782, materially different from the headline academic-argmax recall of 0.719 that the rest of this literature would otherwise cite – Section 6.3, Figure 2.

Takeaway. INT8 quantization loses less accuracy than fp32 export, relocating the real cost to a shared post-processing path – Section 6.3, Figure 3.

The rest of this paper proceeds as follows. Section 2 places this system against the literature it draws on, pivoting explicitly from the domain and deployment literature that earns citations but no novelty claim, through the guardrail and safety-critical-LLM literature that this system’s pattern does not invent, to the specific gap – cross-layer measurement plus a fabrication-inside-agreement finding – that this paper actually fills. Section 3 defines the terms used throughout. Section 4 describes the dual-layer decision system, the vision, retrieval, and MLOps layers, and states the four formal properties the rest of the paper depends on. Section 5 documents how every one of the eleven underlying experiments was actually run. Section 6 is the paper’s evidentiary core. Section 7

and Section 8 state what this paper does and does not license a reader to conclude, and Section 9 closes with the paper’s central claim restated against everything measured in between.

2 Related Work

Two literatures bear on this paper, and they answer different questions. One asks whether the *setting* is real: is precision aquaculture a live engineering problem, and is edge deployment of vision models on Apple Silicon or similar hardware a solved-enough practice to build on. The other asks whether the *safety pattern* is new: is putting a deterministic layer in final authority over a probabilistic one a novel idea. The honest answer to the second question is no, and stating that plainly, with the specific prior art named, is more useful to a reader than pretending otherwise.

2.1 Precision aquaculture and edge vision: an active, competitive setting

Computer-vision monitoring of farmed fish is a mature enough sub-field to have its own taxonomy. Cui et al. [2025]’s survey organizes the literature into detection, counting, behavior classification, and activity analysis, and reports contemporaneous RAS-specific YOLO variants reaching mAP50 above 0.90 – higher than the 0.840 this paper’s own detector achieves on a 510-image, single-class dataset. S.U.R.E.’s vision pipeline is not a vision-accuracy contribution against that baseline, and this paper does not claim it is one. What the field’s own review literature does flag as missing is the closure from a detection to an actionable, safety-graded decision: Fitzgerald et al. [2025] state directly that most deployed vision systems in aquaculture welfare monitoring never complete that trace. Precision-aquaculture reviews describe detectors; they describe welfare taxonomies decomposed into input- and outcome-based indicators [Browning, 2023, Tschirren et al., 2021]; they do not, in general, describe a system that follows one detector’s confidence-threshold choice all the way to the single rule that consumes it. Section 6 traces exactly that path for one rule (`fish_count==0`), and the sturgeon-specific prior work is thin enough – Huang et al. [2024]’s cross-modal stress classifier is the closest, and it is a lab study, not a deployed system – that a working, bench-evaluated, sturgeon-specific decision pipeline is itself a modest but genuine contribution to that narrow corner of the literature, independent of anything claimed about the LLM layer.

The motivating urgency claim – that a dissolved-oxygen crash can kill stock within hours – is not this paper’s own measurement; it is grounded in water-quality literature reporting species- and temperature-dependent lethal thresholds in the 4.1–7.0 mg/L range [Lindholm-Lehto, 2023, AgMRC, 2026], and it should be read with that species/temperature caveat attached rather than as a universal constant. On the edge-deployment side, CoreML-on-Apple-Neural-Engine benchmarking is itself vendor-documented practice, not a novel technique [Ultralytics, 2026], and the INT8 accuracy band this paper reports (−0.0082 mAP50) sits comfortably inside the 0.5–2.0-point range already reported for calibrated quantization in the TensorRT/ONNX literature [Ultralytics Academy, 2025, Karimov et al., 2025]. Community-documented practitioner reports of ONNX and TorchScript export losing accuracy relative to eager PyTorch execution, independent of quantization, already circulate informally on GitHub issue threads [Ultralytics community, 2022, ONNX community, 2024]; this paper’s contribution there (Section 6) is quantifying that suspected mechanism under one controlled, six-configuration sweep against a single held-out validation set, not discovering it. None of this establishes novelty. It establishes that the ground the system stands on – fish detection, tracking with ByteTrack [Zhang et al., 2022], and export-format benchmarking – is real, current, and reasonably well understood, and that S.U.R.E. sits inside it rather than reinventing it. Deployment context matters here too: on-device inference is favored specifically because sub-100ms latency and unreliable rural connectivity make cloud round-trip unattractive [Roboflow,

2025]; this paper’s latency numbers measure model inference only, and should not be compared directly to reported end-to-end field latencies for a cloud-served alternative.

2.2 What this paper does not claim to have invented

The pivot matters enough to state on its own. This paper does *not* claim to have invented deterministic override of a probabilistic component, interlocks, or runtime shielding. That concept is decades old in industrial safety engineering: Safety Instrumented Systems, formalized under IEC 61508 and IEC 61511, already specify a diagnosed, non-bypassable final-authority layer for enumerable hazards, with the entire discipline organized around the assumption that the protected process cannot be trusted to self-correct [AIChE, 2009, IChemE, 2026]. The reinforcement-learning safety literature arrives at the same idea from a different direction: a *shield* vetoes or corrects a learned policy’s proposed action before it reaches the environment, and that literature independently states the same boundary condition this paper’s Formalization 2 derives from first-hand observation of the system – shielding requires a formally specifiable, enumerable safety condition, and offers no guarantee once the thing being checked is open-ended [Corsi et al., 2024, Ferrando, 2025]. Practitioner writing on LLM guardrails has, by 2025–2026, already converged on almost the same slogan this paper’s system embodies – the model proposes, a rule engine disposes [Rulebricks, 2026, Danilec, 2026, Arthur.ai, 2025, Könighofer et al., 2025] – so even the specific framing is not new currency.

One architectural distinction is worth naming rather than glossing over, because it is the one place this paper’s independence claim is weaker than the industrial-safety literature it borrows language from. IEC 61511’s Independent Protection Layer criteria are written around *physical* independence: separate sensors, separate actuators, separate failure paths. S.U.R.E.’s independence is *logical*, at the code level – `backend/rules.py` is a single Python module imported by both the system’s decision path and the evaluation harness, and Section 4 treats that shared-import property as the thing doing the safety work. A shared module that both paths import is a strictly weaker independence guarantee than physically separate hardware, and this paper does not pretend otherwise; it is, however, a real and verifiable property (Section 6 reports a direct object-identity check, not a code-review claim), and it is the property that actually failed once in this system’s own history – an evaluation harness that had silently drifted from the system’s real decision logic, on exactly the scenario (`fish_count==0`) this paper’s vision-safety trace revisits.

2.3 LLM guardrails and agentic reliability: a critical read, not a bibliography

Most published guardrail architectures for LLMs re-validate a model’s output with a second model, a schema check, or a human reviewer – not a dependency-free deterministic rule engine whose escalation-only precedence is enforced by literally sharing code with the system under test. That gap is worth naming before this paper’s own contribution, because it is the specific thing most of this cluster does not do. Academic proposals for LLM-mediated industrial control already exist at the framework or simulation level: Xia et al. [2024] and a related line of work on agentic industrial control [Vyas and Mercangoz, 2025] propose exactly the pattern of routing every LLM proposal through an external validator before actuation. What distinguishes S.U.R.E. from that closest same-domain-class prior art is not the pattern but the evidentiary status: those are proposed or simulated architectures, and this is an implemented system, evaluated offline against synthetically generated sensor data, with a documented near-miss and eleven independently verified experiments behind its numbers. Clinical LLM safety offers a useful contrast in the opposite direction. There, the dominant backstop is human-in-the-loop review, and residual hallucination rates remain non-trivial even under mitigation – one adversarial-robustness study reports failure rates near 44% for

clinical decision support under adversarial prompting [Omar et al., 2025, Basu and Huynh, 2026]. A human reviewer does not scale to a 24-hour, unattended aquaculture deployment; substituting an always-available, unit-tested deterministic module for that human is the design choice this paper measures the consequences of, not a claim that the substitution itself is conceptually new.

On the LLM component’s own reliability, this paper’s negative result for agentic tool-calling (Section 6) sits inside, rather than against, a growing and unresolved literature: small on-device models’ tool-calling reliability is described as an actively contested research area with mixed findings across studies [Sharma and Mehta, 2025], and independent 2025–2026 benchmarks of comparably sized models report similarly severe failure rates [Wannaphaschaiyong, 2026, Tian et al., 2025]. That does not make AQUA-1B’s 0%/0% result uninteresting; it makes it one more literature-consistent data point rather than an outlier demanding a special explanation. The constant-answer artifact this paper flags in AQUA-7B’s tool-selection benchmark is a named, established concern in benchmark-evaluation research more broadly – flagging near-zero-entropy answer distributions as a scoring artifact rather than evidence of discrimination is not new [Balepur et al., 2024] – and this paper’s contribution there is a narrow application of that established discipline to a new task class (closed-menu agentic tool selection), not the invention of artifact-aware evaluation.

The paper’s own fabrication finding – a model reaching the categorically correct status label through a free-text reasoning chain that cites a sensor value never present in the input – sits closest to a strand of hallucination research that most benchmark work cannot reach, because most benchmark work grades only the final answer. General hallucination and confabulation research in natural-language generation is by now a mature survey area [Ji et al., 2023], and a broader taxonomy has recently extended that framing specifically to agentic systems [Lin et al., 2025]. The closest direct match found for this paper’s specific observation – fabrication surviving inside an output whose enumerable field is correct – is a 2026 study on internal representations as indicators of hallucination in agentic tool selection [Healy et al., 2026], which shares the underlying concern but not this paper’s specific empirical instance: a small, self-documented-undertrained local model, in an implemented safety-critical decision loop, where the fabrication is invisible to precisely the evaluation regime – status-only, answer-only grading – that dominates the guardrail and agentic-benchmark literature this cluster otherwise reviews. That is the gap this paper’s Section 6 actually fills, and it is worth stating plainly rather than folding into a longer list: an evaluation regime built to check an enumerable field will not catch a wrong reason attached to a right decision, because it was never designed to look.

2.4 Retrieval calibration and drift-gated MLOps: careful application, not invention

The two remaining clusters ground this paper’s RAG and MLOps components as applications of active, already-established sub-literatures, and this paper treats them exactly that way. Asymmetric retrieval training explains, mechanically, why the deployed e5-small embedding model outperforms a symmetric sentence-similarity model as passage length grows – the query/passage prefix scheme Wang et al. [2022] introduce is the reason, not a novel discovery of this paper. Per-corpus similarity-threshold calibration is itself an established discipline, not a one-off choice: recent work argues explicitly against a universal similarity threshold in favor of corpus-specific precision/recall calibration [Agnihotram and Sarkar, 2025, Braintrust, 2025], and this paper’s 0.85-over-0.84 threshold selection (Section 6) is exactly that discipline applied, with the asymmetric cost stated rather than assumed. It is also bounded by a limit this paper does not try to talk around: recent formal work on embedding-similarity-based hallucination gating shows there are certified limits to how much a similarity threshold alone can prevent [Sinha, 2026, Goswami and Kurra, 2025], and the

overlapping positive/negative similarity distributions this paper measures on its own small corpus independently confirm that caution – a threshold, however carefully chosen, is a mitigation, not an elimination, of retrieval-driven fabrication risk.

Population Stability Index drift detection has the same status: the pitfall this paper’s own MLOps sweep rediscovers – that binning choice materially changes PSI’s sensitivity, and that naive equal-width binning can behave badly under a narrow-banded score distribution – is already documented, standard practice in the credit-scoring literature PSI originates from [Arize AI, 2026b,a, Fiddler AI, 2026, Yurdakul, 2018]. Champion-challenger promotion gating with a minimum-improvement margin before shipping a retrained model is likewise widely documented MLOps practice [DataRobot, 2026, Snowflake, 2026], as is the broader tension between scheduled and drift-triggered retraining [WWT, 2026, Anonymous, 2025]. What is not already documented, as far as this review can establish, is the specific combination this system runs: PSI over YOLO detection- confidence deciles, narrow-banded for a structurally different reason than a credit score is narrow-banded, feeding a champion-challenger gate for a vision-detection registry rather than a tabular classifier. That is a real but modest contribution – an application of two known techniques to a new signal type and a new model class – and this paper states it at that scale, not as a discovery of either PSI or champion-challenger gating.

2.5 What is actually new here

After three clusters of prior art, the honest novelty locus is narrow and specific: the same narrow-enumerable-interface, escalation-only discipline, independently measured across five architecturally distinct layers of one implemented system – decision fusion, agentic tool routing, retrieval, MLOps retraining, and edge vision – including one full disclosed negative result and a fabrication-detection finding the Safety-Instrumented-System and shielding literatures have no equivalent for, because those literatures are built around enumerable action or state spaces, not free-text reasoning that can be factually wrong while the enumerable output next to it is correct. Section 6 is the actual argument for this claim; this section’s job was only to establish that nothing else in the surrounding literature already makes it.

3 Background

This section is deliberately short. Its job is to define the terms the rest of the paper uses and to sketch the system’s shape, not to re-argue the literature placement already made in Section 2.

3.1 Recirculating aquaculture systems

A recirculating aquaculture system (RAS) treats and reuses water within a closed loop rather than drawing continuously from an open source, which makes water chemistry both more controllable and more fragile: a single mechanical or biological failure propagates through the whole tank volume rather than being diluted by fresh inflow. Dissolved oxygen (DO) is the parameter this paper treats as the sharpest deadline. Reported lethal thresholds for farmed fish vary by species and water temperature, roughly in the 4.1–7.0 mg/L range [Lindholm-Lehto, 2023, AgMRC, 2026]; for the sturgeon stock this system monitors, the operative floor is 6 mg/L, and a sustained drop below it can kill stock within hours, not days. That timescale is the reason a monitoring system for this domain cannot tolerate a silent failure mode: there is no grace period in which a human operator is expected to notice and intervene before an LLM’s unnoticed mistake becomes a physical loss.

3.2 The S.U.R.E. system, at a glance

S.U.R.E. is an implemented welfare-monitoring stack, designed to run continuously, built around four communicating services (Figure references and full architectural detail follow in Section 4; this is only an orientation map). A vision service runs YOLOv11s object detection with ByteTrack [Zhang et al., 2022] multi-object tracking over a live or recorded camera feed at roughly 15 frames per second, producing a per-frame fish count and track history. A sensor stream (dissolved oxygen, temperature, pH, and related water-chemistry readings) arrives on a two-second cycle. A backend service fuses both streams, maintains a rolling history, and computes a decision. That decision has two independent sources: a locally hosted LLM (AQUA-1B, a fine-tuned Gemma-3-1B variant with a LoRA domain adapter) that reasons over the fused sensor-and-vision state and produces a natural-language explanation alongside a status label, and a deterministic rule engine that computes the same status label directly from the sensor and vision thresholds, with no dependency on the model at all. A frontend dashboard renders the fused result. A retrieval-augmented generation (RAG) layer supplies the LLM with grounded passages from eight domain-specific knowledge documents when it needs supporting context, and an MLOps layer tracks model registry history and watches the vision model’s own output distribution for drift.

3.3 Definitions used throughout

Severity levels. The system recognizes three ordered severity levels – `ok` < `warning` < `critical` – forming a finite total order. Every decision, whether produced by the LLM or the rule engine, resolves to exactly one of these three labels; there is no partial-severity or confidence-weighted intermediate state exposed to either side of the decision fusion described next.

The rule engine. “The rule engine” refers throughout this paper to `backend/rules.py`, a single Python module with no dependency on the model-serving stack, the web framework, or any request/response object – it is pure functions over sensor and vision values. Both the system’s decision path and the offline evaluation harness import this exact module, a property Section 6 verifies directly rather than assumes. The rule engine’s output is treated as ground truth for what the system’s severity *should* be at any given moment; the LLM’s job is to reason about and explain that state, never to override it downward.

Decision fusion and escalation. When the LLM’s proposed status disagrees with the rule engine’s computed status, the system never lets the LLM’s status stand if it is less severe than the rule engine’s. This escalation-only behavior is stated formally as Formalization 1 in Section 4.

Retrieval-augmented generation (RAG). RAG here means the specific pipeline that embeds incoming queries and eight source documents with the `e5-small` model, retrieves similarity-ranked passage chunks from a `pgvector` store, and supplies retrieved passages to the LLM as grounding context, subject to a minimum-similarity threshold below which nothing is retrieved at all. It is explicitly an enhancement to the LLM’s reasoning, not a dependency the system requires to keep functioning – if the vector store is unreachable, the system continues to operate on the rule engine alone.

Population Stability Index (PSI). PSI is the divergence measure used here to detect drift in the vision model’s own detection-confidence output distribution, computed by comparing a binned reference distribution (captured when the reference window was first established) against a binned

current window using $\text{PSI} = \sum_i (c_i - r_i) \ln(c_i/r_i)$ over bins i with reference and current mass r_i, c_i . The binning scheme used to build those bins is itself the object of one of this paper’s own MLOps findings (Section 6); PSI’s sensitivity is not independent of how its bins are drawn, and that dependency, not merely PSI’s value, is what the paper’s MLOps result is actually about.

4 System Architecture: the LLM-vs-Rule-Engine Paradigm

The central design decision in S.U.R.E. is not any individual model choice. It is where final authority over a safety-relevant decision is allowed to live. This section states that decision precisely, formalizes the two properties it depends on, and then walks the other three subsystems – edge vision, RAG, and MLOps drift gating – that inherit the same discipline at a different layer.

4.1 The dual-layer decision system

Every decision cycle produces two independent status proposals. The LLM, AQUA-1B, receives the fused sensor-and-vision state and returns a status label (`ok/warning/ critical`) plus a free-text `reasoning` field explaining it. Simultaneously, the rule engine (`backend/rules.py`, defined in Section 3) computes its own status label directly from the same sensor and vision thresholds, by a call path that shares no runtime state with the LLM at all. A single function, `apply_rule_override`, then compares the two: if the LLM’s proposed severity is lower than the rule engine’s, the final severity is raised to the rule engine’s value and the rule engine’s reasoning is appended; the LLM’s status is never allowed to override the rule engine downward, and the rule engine’s own decision path is never blocked or delayed by the LLM being slow, malformed, or unavailable.

Formalization 1 (Severity monotonicity (escalate-only invariant)). *Let $\preceq$ be the total order `ok` $\prec$ `warning` $\prec$ `critical` on the system’s finite severity lattice. Writing sev_{rule} for the rule engine’s computed severity and sev_{llm} for the LLM’s proposed severity, the system’s final decision satisfies*

$$\text{sev}_{\text{final}} = \max_{\preceq}(\text{sev}_{\text{rule}}, \text{sev}_{\text{llm}}).$$

This is not a proved theorem; it is a restatement, in one line, of what `apply_rule_override` already does in code, offered as a formal name for the “escalate-only, never downgrade” invariant the rest of the paper refers back to. Its correctness as a safety property depends entirely on one further condition that is not visible in the formula itself: that `backend/rules.py` is the single source computing sev_{rule} everywhere it is consumed, with no independently maintained copy to drift out of sync. That is exactly the property this system’s own history violated once – an evaluation harness once carried its own copy of the rule logic, and the copy disagreed with the system’s real decision path on the single scenario that matters most (`fish_count==0`: the copy said `warning`, the real decision path said `ok`) – and exactly the property Section 6 verifies directly, at the level of Python object identity, not output-matching.

This distinction between *how* a threshold is escalated and *whether* the free-text justification for it can be trusted is the boundary the second formalization exists to name.

Formalization 2 (Enumerable vs. non-enumerable failure modes). *An enumerable failure mode is a finite, coded, checkable predicate over sensor or state variables – for instance, dissolved oxygen falling below 6 mg/L. A non-enumerable failure mode is an open-ended natural-language judgment: a causal narrative that may be internally incoherent or factually wrong about the input it claims to describe, while still landing on the categorically correct status label. Formalization 1’s escalate-only override is a sound, verifiable safety net for the former, by construction – it only ever reads the enumerable*

status field. It provides no protection whatsoever against the latter, because `apply_rule_override` never inspects the *reasoning* field at all. This distinction is not a hedge added after the fact; it bounds exactly what Formalization 1 is entitled to claim, and Section 6 reports the first concrete, in-system evidence that both failure classes actually occur on this system’s own scenario set, not merely that they are conceivable.

AQUA-1B and adapter provenance. AQUA-1B is a Gemma-3-1B-class model fine-tuned with a LoRA domain adapter for this system’s Turkish-language reasoning task. It is worth stating plainly, at first mention, what that adapter actually is: the adapter loaded in the running configuration (`sure-aqua-adapter`) was trained on eight handwritten examples, and the project’s own fine-tuning script documents this exact dataset as insufficient – “8 handwritten examples, smoke-test only” – while a properly sized, 128-example alternative dataset sat unused on disk, predating this adapter’s own training run. This is disclosed here, not as an apology, but as the honest scope of what is measured: every AQUA-1B result reported for the decision layer in this paper (Section 6) reflects this smoke-test adapter, not the untested larger one. Framed as a stress test rather than a limitation, this is the more informative condition to have measured first – if the architecture’s safety property holds against a model given close to no fair chance to reason well, that is stronger evidence for the property’s independence from model quality than a result obtained against a carefully tuned model would have been. This scope note applies strictly to results derived from this adapter (Section 6’s decision-layer findings); it does not apply to the agentic tool-routing results reported in the same section, which use the base model with no adapter loaded at all.

4.2 Edge vision pipeline

Detection runs YOLOv11s over each incoming frame, with ByteTrack [Zhang et al., 2022] maintaining per-fish tracks across frames so that a single occluded or re-detected fish is not double-counted. The detector was trained on 510 labeled images (412 train / 98 validation, single class `sturgeon`). These are real photographs, captured from the project’s own physical camera and system setup and hand-labeled by the team – not synthetic or scraped imagery – but the dataset is intentionally small at this stage, and expanding it is planned as future work, not treated here as already sufficient. The detector runs via CoreML on the Apple Neural Engine for on-device inference; Section 5 details the training and export methodology, and Section 6 reports the accuracy and latency measurements across every export format this system was benchmarked on. The configured confidence threshold that gates which detections are kept is a separate design choice from the threshold that maximizes F1 on a held-out curve, and the two do not coincide here – a fact this paper’s own re-measurement surfaced rather than assumed.

Formalization 3 (Vision operating-point tradeoff framing). *A detector’s configured confidence threshold selects one specific point on its precision–recall curve. That point is, in general, distinct from the threshold that maximizes F1 on a held-out curve – the two coincide only by construction or by coincidence, never by default. For a downstream binary safety rule that consumes recall directly (here, `fish_count==0` triggering a warning), the quantity that governs operational risk is the recall at the configured threshold, not the recall at the academic-style F1-argmax that a validation report typically headlines. Section 6 shows this is not a merely theoretical distinction for this system: the two numbers differ materially (0.782 versus 0.719), and citing the wrong one understates, not overstates, the system’s safety margin.*

4.3 Retrieval-augmented generation

The RAG layer indexes eight domain-specific Turkish-language documents in `pgvector`, chunked by document heading rather than by a fixed word count, and embedded with `e5-small` [Wang et al., 2022]. A query is answered by retrieving the highest-similarity chunks above a minimum-similarity threshold and supplying them to the LLM as grounding context with an inline citation marker; below the threshold, nothing is retrieved and the model reasons without external context. The threshold is set to 0.85 in the running configuration, one step above the empirical F1-optimum of 0.84 (Section 6), a deliberate choice to favor precision – suppressing a marginal, possibly-irrelevant retrieval entirely – over recall, because a fabricated, confidently cited context is judged more costly in this domain than a missed one the rule engine can still act on independently. Tool routing for the agent layer described in Section 6 is deterministic code, not LLM reasoning, for the same underlying reason: correctness of *which* tool is called is treated as an enumerable decision the system should not delegate to a model whose tool-selection reliability has not been established.

4.4 MLOps and drift-gated retraining

Because frames processed at inference time carry no ground-truth labels, drift in the vision model is inferred from the model’s own output: the distribution of detection confidence scores, compared against a reference distribution captured when the reference window was first established, via the Population Stability Index (Section 3). A drift signal alone only says the world changed; it says nothing about whether a retrained candidate would actually be better, so a second, independent gate stands between a drift alert and a registry update.

Formalization 4 (PSI three-way gate). *Given a reference distribution, quantile-edge (decile) binning of that reference, and a PSI value computed over a sliding current window against those bins, the system maps the resulting PSI value to a three-way decision $\{\text{none}, \text{review}, \text{retrain}\}$ at fixed thresholds ($\text{MODERATE}=0.10$, $\text{SIGNIFICANT}=0.25$). A **retrain** decision does not by itself promote a new model: any retrained candidate must additionally beat the incumbent by more than a fixed MIN_IMPROVEMENT margin (0.005 mAP50, absolute) before it is registered, a guard against shipping a delta indistinguishable from ordinary training noise. Section 6 reports both halves of this gate exercised directly – the drift decision and the promotion gate – and reports a genuine correction to how sensitively the two candidate binning schemes (quantile versus equal-width) actually behave across a severity sweep, rather than assuming either convention’s textbook behavior.*

The choice between quantile and equal-width binning is not a cosmetic implementation detail: the detector’s confidence scores cluster narrowly (roughly 0.5–0.9), so a small number of equal-width bins can each hold a large, uneven share of the reference mass, and Section 6 traces exactly how that structural property changes PSI’s sensitivity as a function of drift severity.

5 Experimental Setup and Edge Metrics

All methodology described in this section – sample sizes, hardware, and measurement procedures – refers to offline, bench-level evaluation run against synthetically generated sensor data, static labeled images, and scripted or synthetic scenarios executed against the codebase. None of it refers to live sensor telemetry from an operating recirculating aquaculture facility; no such facility or live sensor feed was used anywhere in this paper, and no physical water-quality sensor hardware exists for this system at all – every dissolved-oxygen, temperature, and other water-chemistry value referenced anywhere in this paper, including in Section 6’s fabrication analysis, was synthetically

generated for a scripted test scenario, not recorded from a real sensor. This is a deliberate scope choice, not an incidental one: absent physical sensor hardware, the purpose of this evaluation is to demonstrate the architectural feasibility of running this system’s multi-component pipeline – LLM, vision, retrieval, and rule engine together – and specifically to test whether its space and memory footprint is viable on realistic edge hardware, rather than to validate decision-making accuracy against a live water body. Every number in Section 6 comes from one of eleven experiments run directly against the S.U.R.E. codebase’s own already-running local configuration at git commit `3c1b9fa`, not from a synthetic benchmark constructed to flatter the architecture. This section states, honestly, what each experiment measured, on what sample, with what hardware, and what its own stated limitations are – the same discipline the paper asks of its own headline numbers is applied here to how they were obtained.

5.1 General methodology and guardrails

All experiments ran read-only against the running codebase and, where applicable, the live `sure_rag` database: no experiment inserted, updated, or deleted a database record, and no experiment modified `sure-project` source files. Three experiments (RAG calibration, vision leakage, and export benchmarking) required new, purpose-built analysis scripts rather than reusing an existing one, because the existing scripts either wrote to a live database (`rag/bench.py`) or the needed diagnostic did not exist yet (perceptual-hash leakage detection, per-image ONNX-versus-TorchScript agreement). Every script and its raw output is preserved alongside its experiment’s `results.md` file for audit. Hardware for every vision and edge-latency measurement was a single Apple M4 Pro; no claim in this paper generalizes across device classes, and none is presented as if it did.

5.2 Decision-fusion audit and behavioral trace (G2, G3)

The claim that the evaluation harness and the system’s decision path compute severity identically was checked at the strongest available level: not by comparing outputs, but by confirming, via Python’s `is` identity operator, that both paths import and hold a reference to the exact same in-memory `rules` module object. This was verified across all 8 of the evaluation harness’s built-in scenarios, with 8/8 agreement.

The dual-layer decision system’s actual behavior was then traced across those same 8 scenarios ($n = 8$, the harness’s entire fixed scenario population – this is not a subsample, and no confidence interval is implied by or should be read into this count). For each scenario, the real, unmodified `apply_rule_override` function was invoked against AQUA-1B’s live output and the rule engine’s independently computed status, and each of the 8 outcomes was classified into one of four mutually exclusive buckets (unparseable-defaulted-to-ok, parseable-and-agrees, parseable-and-under-calls-and-escalated, parseable-and-over-calls). A separate mechanism-correctness check (`backend/test_decision.py`, 18/18 passing under synthetic fixtures) was run and is reported strictly apart from this behavioral trace, per this project’s own methodological discipline: a unit test confirming the override logic is implemented correctly is not evidence about how the real model’s output actually distributes across that logic’s branches, and the two are never substituted for one another anywhere in this paper. All 8 free-text reasoning strings were additionally hand-coded for factual accuracy against the true sensor snapshot each scenario supplied – a qualitative, single-rater judgment, reported as illustrative rather than a statistically powered claim, but checkable in principle by any reader against the same 8 published snapshots.

5.3 Agentic tool-routing benchmark (G4)

The agentic tool-calling benchmark ran two models (AQUA-1B and AQUA-7B, base weights, no LoRA adapter loaded for either) against a closed menu of three read-only tools, first reproducing the original 5-scenario benchmark, then extending it with 4 newly authored scenarios designed specifically to stress a different failure mode than the original 5 already covered (a different sensor parameter’s trend, a recovering rather than degrading trend, a combined vision-and-sensor cue, and a borderline-but-safe restraint case), for a total $n = 9$ scenarios $\times$ 2 models = 18 independent agent runs. Greedy decoding (temperature 0) was used throughout, so no scenario was run by simple repetition – reported effect sizes are entirely a function of scenario diversity, not resampling the same input.

5.4 Vision dataset leakage audit (G5)

Cross-split leakage between the 412-image training set and 98-image validation set was checked two ways: an exact frame-index overlap check (trivial to compute, and confirming zero exact duplicates), and a perceptual near-duplicate check using 64-bit average-hash and difference-hash fingerprints computed for every image (the `imagehash` library was unavailable in this environment; a minimal equivalent was implemented directly from PIL and NumPy). A known-positive control – a separate, already-documented dataset with training and validation splits pointing at the identical folder – was run through the same pipeline first, to confirm the method actually detects a leak it is known to contain, before trusting a null or partial result on the real split.

5.5 Export-format benchmark and edge latency (G6)

Six export configurations of the same trained checkpoint (`best.pt`, epoch 77) were benchmarked against the identical 98-image validation split: native PyTorch on both MPS and CPU, ONNX (fp32 and INT8), CoreML (dispatched to the Apple Neural Engine), and TorchScript. Latency was measured as p50/p95 over pre-decoded validation frames (avoiding disk I/O inside the timed loop), discarding warm-up runs, following the same protocol the system’s own export benchmarking script uses by default. Because a single latency session can be affected by thermal state or scheduler noise, the CoreML/ANE configuration – the one actually configured for use – was re-measured across three independent process invocations rather than reported from a single run. A new per-image diff between the ONNX and TorchScript exports’ detections (greedy IoU-matched, 0.9 IoU bar for agreement) was run across the full 98-image set to test directly whether their shared accuracy loss reflects a shared mechanism rather than independent numerical noise.

5.6 Vision operating-point safety trace (G7)

The configured detection confidence threshold was confirmed directly from the vision service’s own source (`CONF_THRESH=0.20`) rather than assumed from documentation. Two complementary methods were used to characterize precision and recall as a function of confidence: sweeping `model.val(conf=X)` over a coarse grid, and separately reading Ultralytics’ underlying fine-grained (1000-point) precision/recall/F1 curve directly from a single validation call. The first method was found, over the course of this audit, to have a methodological trap worth recording for other practitioners: it reports an argmax-over-the-retained-curve figure regardless of the requested confidence, so it cannot answer “what is precision/recall *at* this specific threshold” – only the second, curve-reading method can, and it is the one this paper’s headline numbers rely on. The full-frame-miss

rate (a frame producing zero detections despite containing ground-truth fish) was measured empirically across all 98 validation frames at the configured threshold, and separately estimated via an independence approximation $P(\text{miss} \mid k) = (1 - \text{recall})^k$ for comparison. Because the validation set’s minimum ground-truth fish-per-frame count is $k = 3$ – it contains zero frames with $k = 1$ or $k = 2$ fish – the sparse-frame regime most relevant to a welfare check involving a small or isolated fish could only be estimated by extrapolation, never measured directly; this is reported as a dataset-coverage gap, not folded silently into the headline result.

5.7 PSI drift-binning sweep (G8)

A 16-point severity sweep ($\delta \in \{0.00, 0.02, \dots, 0.30\}$) was constructed by resampling the reference confidence distribution ($n = 1467$) with replacement and applying a mean shift of δ , clipped to $[0, 1]$, then computing PSI under both quantile-edge and fixed equal-width binning at every severity level. This is a mean-shift-only perturbation model; variance inflation was not layered in and is noted as a natural extension rather than attempted here. The three-way `decide()` gate and the promotion `gate()` function were each additionally exercised directly against two hand-constructed synthetic windows (no-drift and significant-drift) and three hand-constructed promotion-margin cases, to confirm the gate’s documented behavior end-to-end.

5.8 twin_bridge mechanism-evidence inventory (G9)

The existing PLC/CODESYS bridge test suite (`twin_bridge/test_bridge.py`) was run as-is, and four new edge-case tests were authored against a scripted `FakeTwin` test double to exercise a comparison-logic branch the existing suite never reached. No live Godot or CODESYS session was available in this environment, and none was constructed for this audit; every test in this subsystem, existing and new, is team-authored mechanism evidence run against a scripted double, not evidence from an independently produced external system.

5.9 RAG threshold calibration (G1) and manuscript-wide provenance audit (G10, G11)

The RAG threshold sweep and the e5-small-versus-`tr-bert` retrieval comparison were reproduced read-only against the live 44-chunk indexed corpus. Because the original benchmark script clears and re-populates the live collection (a write operation forbidden under this audit’s read-only guardrail), a new script reproduced only the query-and-scoring half of that benchmark against whatever was already indexed, after first confirming the indexed content matched the current knowledge-base files exactly. A new random-retrieval baseline (uniform selection over all 44 chunks, evaluated across 5 seeds for stability) was authored specifically to establish that e5-small’s measured retrieval skill is a real effect and not an artifact of the corpus’s small size. Separately, a full manuscript-wide provenance and stale-figure audit re-verified every headline number cited in this paper against its raw source file and git commit, and re-ran a repository-wide grep for the specific superseded recall figure (0.695) this project’s own history required guarding against; that audit is what surfaced the still-unfixed occurrence of that figure inside the system’s RAG knowledge base reported in Section 6.

6 Results

Eleven pre-registered experiments were run against the system’s own codebase, each independently verified against its own raw output. Eight reached their strongest pre-registered success criterion;

Table 1: Per-goal achievement summary against each goal’s own pre-registered strong/minimum-viable criteria. No goal reached `partially_achieved`, `not_achieved`, or `blocked`. Source: `formalized_results.json`.

Goal	Experiment	Verdict	Headline finding
G1	EXP01	strong	RAG threshold: precision-favoring, argmax reproduced, random-baseline margin
G2	EXP02	strong	eval.py/runtime rule path is the identical module object (8/8)
G3	EXP03	minimum-viable	fail-safe defaulting dominates (50%); fabrication survives agreement
G4	EXP04	strong	agentic tool-routing negative result, replicated at $n=9$
G5	EXP05	minimum-viable	32 near-duplicate frame pairs; corrected metric range now quantified
G6	EXP06	strong	export-format loss mechanism; INT8 loses less than fp32
G7	EXP07	strong	configured-threshold recall (0.782) $\neq$ headline argmax recall (0.719)
G8	EXP08	minimum-viable	PSI binning sensitivity crossover, not confirmed monotonic
G9	EXP09	strong	twin_bridge: unit-tested, never live-validated
G10	EXP10	strong	no closer prior instance of the cross-layer thesis found
G11	EXP11	strong	manuscript-wide audit; found a live, unfixed stale figure

three (the dual-layer behavioral trace, the vision leakage audit, and the PSI drift-binning sweep) reached only their pre-registered minimum-viable criterion, because each of them found a genuine complication rather than a clean confirmation. Table 1 gives the full scorecard before the narrative below. None of the three minimum-viable results is written around here as a shortfall – they are, in order, the paper’s central finding, a real and now-quantified data-integrity correction, and a mechanistically explained reversal of a naive expectation.

6.1 The dual-layer decision system: fail-safe defaulting, not correction

The claim that the evaluation harness and the system’s own decision path compute severity identically was verified at the strongest available level: both resolve to the exact same in-memory `rules` module object (a Python `is` identity check, not output-matching), with 8/8 scenario agreement. This closes, structurally, the specific failure mode that motivated building this check in the first place – a copied rule engine silently drifting from the system’s real decision path.

That structural guarantee only tells us the two paths compute the *same* severity given the same inputs. It says nothing about what AQUA-1B’s actual output looks like, or how often the escalation it is supposed to trigger actually fires. Recall that every result in this subsection was obtained against the configured `sure-aqua-adapter`, trained on eight handwritten examples that the project’s own fine-tuning script documents as insufficient, smoke-test-only material – not the properly sized, 128-example alternative that sat unused on disk before this adapter was even trained (Section 4.1). That is stated here again, deliberately, because it is the fact that changes how the following result should be read.

One might expect a dual-layer safety system’s value to come from the rule engine catching

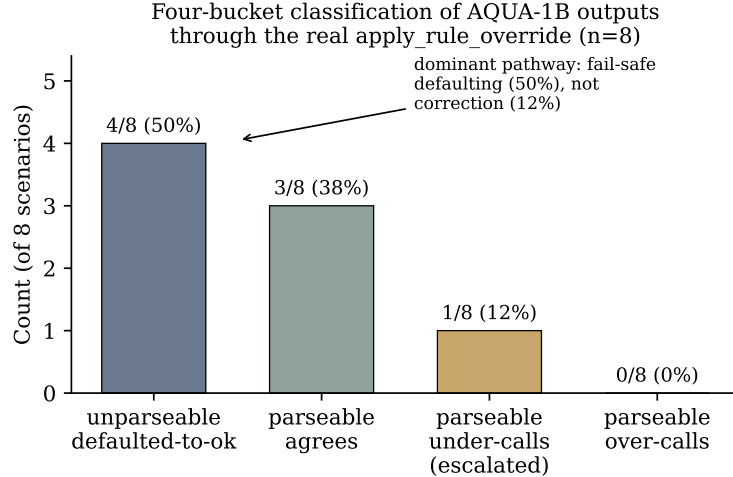


Figure 1: Four-bucket behavioral classification of AQUA-1B’s output against the real, unmodified `apply_rule_override` path, across all 8 evaluation-harness scenarios ($n = 8$, single local 1B model with an 8-example smoke-test adapter, single rater for the reasoning-content coding pass). The dominant observed pathway is malformed-output fail-safe defaulting (50%), not sophisticated severity correction (12%); zero over-calls were observed, but $n = 8$ is too small to conclude the over-call bucket is rare in general.

the model’s occasional wrong call. It rarely gets the chance to: half the time (4/8) the model’s output does not parse at all, and – more surprising still – even when its final status agreed with the rule engine, in two of those three cases the reasoning behind it cited a sensor value that was never in the input. Figure 1 shows the full four-bucket distribution across all 8 of the evaluation harness’s scenarios ($n = 8$, the entire fixed scenario population; no confidence interval is implied): 50% unparseable-defaulted-to-ok, 38% parseable-and-agrees, 12% parseable-and-under-calls-and-escalated, and 0% parseable-and-over-calls (too small an n to conclude over-calls are rare in general – only that none occurred here). Manual, single-rater coding of all 8 free-text reasoning strings against each scenario’s true, synthetically generated sensor snapshot found 6 of 8 materially misrepresent that snapshot – of those, 3 (T02, T05, T08) fabricate a specific numeric sensor value never present in the input (stating, for instance, dissolved oxygen 6.0 mg/L against the scripted test scenario’s actual input value of 5.7 – itself a synthetically generated test value, not a live reading), and the remaining 3 are inaccurate in a different way – an unsupported blanket claim with no invented number (“all parameters optimal” during an active multi-parameter emergency) or an internally incoherent reading of a plausibly real value. In two cases (both inside the ostensibly cleanest “agrees” bucket) the categorically correct final status is reached via an entirely invented causal narrative built on a fabricated number. Whenever that 38% agrees figure is quoted anywhere in this paper, this same fabrication rate belongs beside it: the number is not a measure of the model reasoning correctly two-thirds of the time it agreed with the rule engine, because in two of three such cases it did not.

We did not give the LLM a fair chance – the configured adapter trained on 8 handwritten examples the project’s own training pipeline calls insufficient, while a documented, properly sized 128-example alternative sat unused on disk – and the architecture’s safety property held anyway. Stated without hedging, because it should not read as an apology: this makes the finding above stronger evidence for the architecture’s design, not weaker. A model given close to no fair chance

to reason well still could not push the system’s final severity below what the rule engine alone would compute in any of the 8 scenarios, including the one case (T02, a critical dissolved-oxygen scenario) where its own proposed status was wrong. The safety property measured here never depended on AQUA-1B reasoning correctly. It depended on one enumerable field being extractable from whatever the model produced, and on the system defaulting to a safe state when it was not – exactly Formalization 2’s boundary, now with a concrete, in-system instance rather than a theoretical one. `backend/test_decision.py`’s own mechanism-correctness suite (18/18 passing under synthetic fixtures) confirms the override logic itself is implemented correctly; it is reported here strictly apart from the behavioral finding above, because a passing unit test says nothing about how a real model’s output actually distributes across that logic’s branches.

6.2 Agentic tool-routing: a negative result that survives an adversarial re-run

The adapter-provenance caveat above does not apply to this subsection: both models here ran base weights, with no adapter loaded at all. AQUA-1B never produces a parseable tool call under the system’s closed-menu agent loop – 0% format compliance, 0% tool-selection accuracy, on both the original 5-scenario benchmark and a deliberately adversarial re-run designed to break the result rather than confirm it. That re-run added four newly authored scenarios the model had never seen, built specifically to test whether AQUA-1B’s failure is prompt-specific: it is not. AQUA-1B’s 0%/0% failure holds identically on all four new scenarios, for $n = 9$ total.

AQUA-7B, the larger model, holds the response format more often but chooses the same tool, `get_sensor_trend`, as its first action in every one of 9 genuinely independent scenarios – including two scenarios where that tool is not among the acceptable answers at all. This is a constant answer, not evidence of discrimination between scenarios, and the benchmark’s own constant-answer check correctly flags it as such rather than letting the raw selection percentage (which nominally rose from 50% to 71.4% on the larger scenario set, an artifact of three of the four new scenarios happening to be sensor-trend-relevant by design) stand uncontextualized. We tried to break this finding by growing the scenario set and it got stronger, not weaker: nine independent samples of a model emitting the same answer regardless of scenario content is harder to explain away as a small-sample coincidence than five. One disclosed discrepancy against the system’s own published benchmark table: format compliance and mean step count for AQUA-7B did not reproduce exactly on this re-run (100%/2.0 steps measured here versus a previously published 60%/3.6), while the selection percentage and the constant-answer conclusion reproduced exactly; no configuration difference was found to explain the discrepancy, and it is reported as an unexplained, non-conclusion-changing drift rather than silently reconciled.¹

6.3 Edge vision: an honest re-measurement, and a counter-intuitive export finding

The system’s detector (YOLOv11s, trained on 510 labeled images, single class `sturgeon`) is reported at mAP50 0.840, precision 0.858, recall 0.719.² These are the numbers a validation report headlines by convention: the confidence threshold that maximizes F1 on the held-out curve. They are not, however, the numbers the system’s configured threshold actually produces.

¹Most likely explained by version drift in the underlying `mlx-lm`/model-serving stack between the original benchmark run and this reproduction; not investigated further, as it does not affect either model’s negative result.

²A fresh reproduction of `model.val()` on the exact configured checkpoint (`best.pt`, epoch 77) in this audit returned precision 0.859 and mAP50 0.8395 – a thousandths-place rounding difference against the reported 0.858/0.840, not a new discrepancy; recall reproduced exactly at 0.719. Table 3, row 3, records this for completeness.

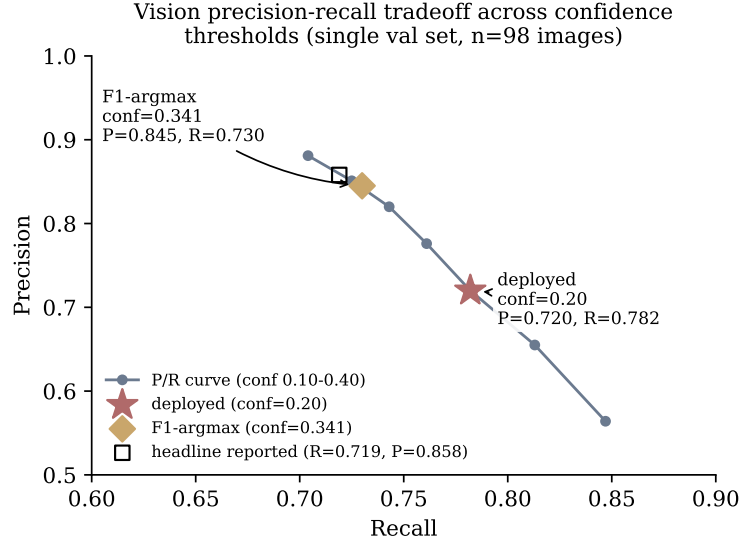


Figure 2: Precision–recall tradeoff across confidence thresholds (single validation set, $n = 98$ images, single device). The academic-style F1-argmax is functionally identical to the headline reported metric ($\Delta F1 = 0.0008$), but the system’s configured operating point differs: the confidence threshold trades precision for recall, actually operating at $R = 0.782$.

Formalization 3 named this distinction in the abstract; here it is concrete. The configured confidence threshold (`CONF_THRESH=0.20`, confirmed by direct reading of `yolo_runner.py`, deliberately lowered from 0.30 for recall on crowded frames) selects a different point on the precision–recall curve than the F1-argmax. Figure 2 plots both.

Disclosure

At the configured threshold, the detector actually runs at precision 0.720 / recall **0.782** – lower precision, *higher* recall than the headline $P=0.858/R=0.719$ the rest of this paper otherwise cites (functionally the true F1-argmax, at $\text{conf} \approx 0.341$, differing by only 0.0008 in F1). Both numbers are correct; they answer different questions. For any claim about full-frame-miss risk under the system’s configured operating point – the quantity that feeds the downstream `fish_count==0` safety rule – **0.782 is the number that governs it**, not 0.719. This is more reassuring than a naive reading of “recall 0.719” suggests, not less, and it is stated here explicitly because the manuscript-wide provenance audit (Section 6.6) found no existing document distinguished the two numbers.

Measured directly against all 98 validation frames, zero produced no detections at all at the configured threshold (0/98 full-frame misses), consistent with an independence approximation that predicted an expected 0.053 misses over the same set. This result carries a coverage gap that belongs beside it, not in a separate limitations paragraph read later: the validation set’s minimum ground-truth fish count per frame is $k = 3$; it contains *zero* frames with $k = 1$ or $k = 2$ fish. The full-frame-miss regime most relevant to a genuinely sparse or isolated fish – exactly the case a welfare check most needs to catch – was never designed into this dataset and cannot be measured from it. Extrapolating the same independence approximation to $k = 1$ suggests a miss probability of roughly 22–28% depending on which recall figure is used; this is an untested extrapolation, stated

Table 2: Headline metrics across the four measured layers, with the G5-corrected vision range shown alongside the uncorrected headline where applicable. The correction is a same-checkpoint re-evaluation on a smaller held-out set, not a leakage-free retrain.

Metric	Headline	G5-corrected	Source
Vision mAP50 ($n=98$)	0.8395	0.8335–0.8345 ($n=81-85$)	EXP06/EXP05
Vision precision	0.858–0.859	0.846–0.852	EXP06/EXP05
Vision recall (F1-argmax)	0.719	0.712	EXP07/EXP05
Vision recall (<i>configured</i> , conf=0.20)	0.782	—	EXP07
Edge latency, CoreML/ANE p50/p95	9.03/9.60 ms	—	EXP06
RAG: MRR (e5-small/heading) vs. random	0.856 vs. 0.418	—	EXP01
RAG: hit@1 vs. random	0.793 vs. 0.207	—	EXP01
INT8 export Δ mAP50 vs. fp32 ex- port	−0.0082 vs. −0.0104	—	EXP06

as one, not a measurement.

A separate audit of the train/validation split found 32 near-duplicate frame pairs at a strict perceptual-hash threshold, 14 of them immediately adjacent frame indices of the same source video – a real, previously undocumented leakage risk, milder than and distinct from an already-corrected exact-duplicate case elsewhere in this dataset’s history. A same-checkpoint re-evaluation on the validation set with the affected frames excluded (not a leakage-free retrain: the flagged training frames remain in the set that produced this checkpoint’s weights) gives a corrected range of precision 0.846–0.852, recall 0.712, mAP50 0.8335–0.8345, depending on how conservatively the exclusion is drawn – a real but modest 0.5–1.3 point optimistic bias, smaller than the dataset’s already-known leakage case and not conclusion-changing. Table 2 reports both the uncorrected and corrected ranges together, and this caveat sentence should travel with the headline vision numbers wherever they are cited outside this paper as well.

Export-format benchmarking across six configurations of the same checkpoint (Figure 3) reproduces the configured CoreML/ANE latency figure as genuinely stable across sessions, not a single lucky sample: p50 9.03 ± 0.22 ms, p95 9.60 ± 0.18 ms across three independent process invocations, nearly four to five times faster than any CPU-based export while costing under one accuracy point relative to the fp32 baseline. One might expect the more aggressively compressed format to pay the larger accuracy cost. INT8 quantization loses *less* (Δ mAP50 −0.0082) than the unquantized fp32 ONNX and TorchScript exports already do (−0.0104) – the loss lives in a shared export/post-processing path, not in the numeric precision that gets blamed by default. A new per-image diff between the ONNX and TorchScript exports found 0 of 98 validation images meaningfully different (median matched-pair IoU 0.99999976), direct evidence that the two exports’ shared loss is systematic rather than independent numerical noise, though this diff shows the two exports agree with *each other*, not a full isolation of which specific stage of the shared post-processing path is responsible.

6.4 RAG: an asymmetric-cost threshold, not a raw F1 optimum

The configured retrieval threshold (0.85) is one step above the empirical F1-optimum (0.84, F1 0.951 versus 0.906 at the configured threshold), trading 17.2 points of recall for zero negative-retrieval leakage (precision 1.000 at 0.85 versus 0.906 at 0.84). Figure 4 shows the full sweep; the positive

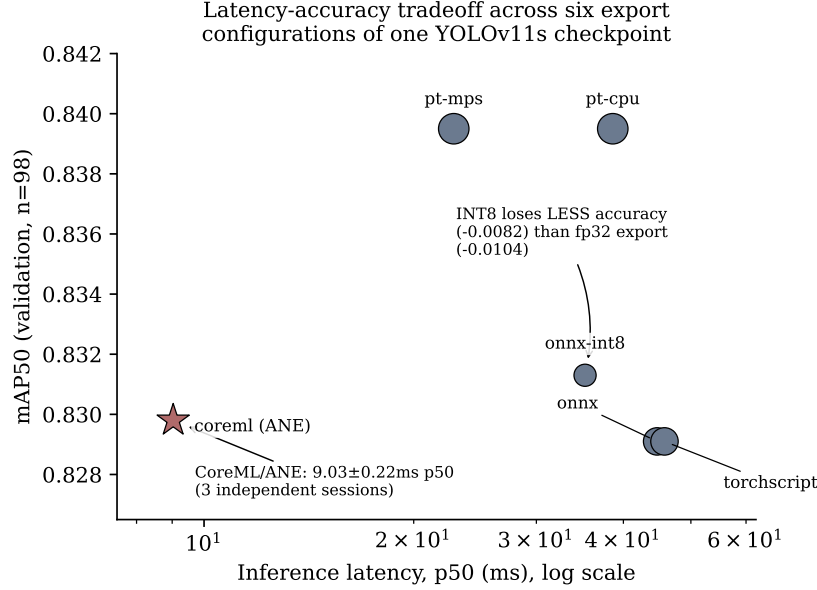


Figure 3: Latency–accuracy tradeoff across six export configurations of the same YOLOv11s checkpoint (single validation set, $n = 98$ images, single Apple M4 Pro device; latency is model-inference-only, not representative of network round-trip in a cloud-served alternative). CoreML/ANE (starred) is both the fastest and within one mAP50 point of the fp32 baseline.

and negative similarity distributions genuinely overlap on this corpus (negative max 0.847 exceeds positive min 0.841), so no threshold fully separates them, and the model’s own instruction to decline an unanswerable query is doing real, unmeasured work a similarity cutoff cannot substitute for. This is a deliberate, disclosed asymmetric-cost choice, not an unexamined default: a missed document only weakens reasoning the rule engine still adjudicates independently, while a passed negative hands the model irrelevant material to blend into a confidently cited, wrong answer. A new random-retrieval baseline (uniform selection over the corpus’s 44 chunks, stable across 5 seeds) confirms the configured retriever’s measured skill is real semantic retrieval and not a small-corpus artifact: hit@1 margin +0.586 (0.793 versus 0.207), MRR margin +0.438 (0.856 versus 0.418) over random chance.

6.5 MLOps and drift detection: a crossover, not a confirmation

A 16-point severity sweep over quantile and equal-width PSI binning does not confirm the naive expectation that equal-width binning misbehaves early and quantile binning grades smoothly throughout. It confirms something more specific and, on inspection, more useful. Figure 5 shows quantile binning is *more* sensitive than equal-width at small early shifts ($\delta \leq 0.06$: at $\delta = 0.02$, quantile PSI 0.164 already reads “moderate” while equal-width’s 0.078 still reads “none”), and only from $\delta \geq 0.08$ onward does equal-width overtake and grow more explosively (at $\delta = 0.30$: 7.649 versus 4.844). Per-bin decomposition traces the mechanism directly: the detector’s confidence scores cluster in a narrow band, so one large fixed equal-width bin holds roughly 30% of the reference mass; once a shift empties that bin, its own log-ratio term alone dominates the equal-width total. Quantile binning, with no single bin holding more than about 10% of reference mass by construction, has no comparable single point of failure. The correct summary is a crossover with a mechanistic explana-

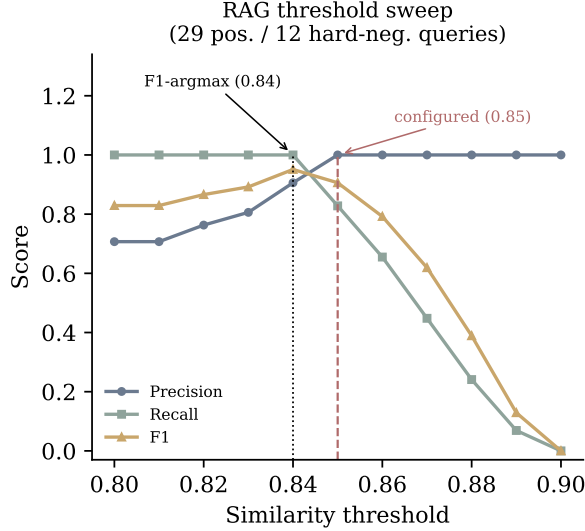


Figure 4: Precision/recall/F1 sweep over the RAG similarity threshold (single 8-document/44-chunk corpus, 29 evaluation queries). The configured threshold (0.85) sits one step above the F1-optimum (0.84), a deliberate precision-favoring choice.

tion, not “quantile binning is more sensitive, use quantile” – at $\delta \geq 0.08$ the relationship reverses, and a practitioner choosing a binning scheme from this result alone would need to know which severity regime their own drift is expected to occur in.

The three-way retraining gate this sweep feeds was exercised directly against two synthetic windows and behaved exactly as designed: a no-drift window (PSI 0.0048) yielded a **none** decision, and a significant-drift window (PSI 0.9070) yielded **retrain**. The promotion gate itself correctly rejected a candidate that failed to beat the incumbent, correctly rejected a candidate whose improvement (+0.0025 mAP50) sat inside the fixed noise band, and correctly accepted a candidate that cleared it (+0.015 mAP50) – all three outcomes matching the gate’s documented behavior exactly.³

6.6 Auxiliary evidence and manuscript-wide provenance audit

The system’s third-party digital-twin comparison module, **twin_bridge**, is designed and unit-tested – its existing test suite passes 18 of 19 cases (the one failure traces to a float-truncation bug in a test helper, not the actual decoding logic it tests), and four new edge-case tests targeting a previously unreachable comparison branch all pass – but it has never been exercised against a live Godot or CODESYS session, and no such session exists anywhere in this project’s history. Every test in this subsystem, existing and new, is team-authored mechanism evidence run against a scripted double, not evidence from an independently produced external system, and it is described that way throughout this paper: unit-tested, never field-validated.

A manuscript-wide audit re-verified every headline number in this paper against its raw source file and git commit, and re-ran a repository-wide grep for a specific recall figure (0.695) this project’s history required guarding against, after an earlier mislabeled-checkpoint error was corrected. That

³One internal inconsistency, self-contained and non-conclusion-changing: the sweep’s own $\delta = 0.04$ entry (quantile PSI 0.7279) differs from a separately drawn significant-drift window at the same nominal δ (PSI 0.9070), an artifact of unseeded per-call resampling inside an otherwise fixed-seed script. No headline number in this paper depends on either individual value.

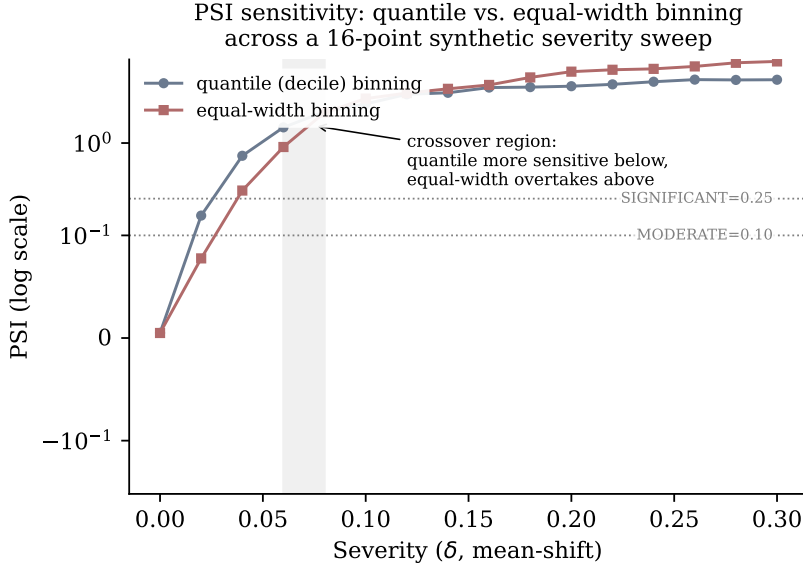


Figure 5: PSI computed via quantile (decile) versus equal-width binning across a 16-point synthetic mean-shift severity sweep on detection-confidence scores ($n = 1467$ per window). Equal-width binning is *less* sensitive than quantile at small early shifts but becomes more explosive once a single large fixed bin empties, confirmed by per-bin decomposition.

grep passes everywhere the figure could reach a reader of this paper – but it surfaced something outside the paper’s own text: the system’s RAG knowledge base itself still states, as a live fact, that the detector’s recall is “~0.695,” 43 minutes’ worth of commits behind the correction applied everywhere else, and unfixed at the time of this audit. This is not a citation error a reader of this paper could ever encounter directly. It is a retrievable, user-facing stale claim inside the system’s own retrieval corpus – the exact hazard a fabrication-and-disclosure discipline paper should least want to find inside its own codebase, and it is reported here rather than quietly fixed and never mentioned. Table 3 lists this alongside five smaller inconsistencies the same audit found, in descending severity.

7 Discussion

The results in Section 6 support a narrower claim than the architecture’s original design intuition promised, and the narrower claim is the more useful one. “The rule engine catches the model’s mistakes” turns out to describe only 12% of what actually happens; “the system defaults to a safe state whenever the model’s output cannot be trusted, and never lets a trusted output fall below what the rule engine alone would say” describes all of it. That the first, catchier framing is wrong is not a weakness in the finding – it is the finding. A reader who walks away believing the rule engine spends most of its time correcting nuanced severity misjudgments would misunderstand what this system’s safety property actually rests on, and would likely misjudge how well the same pattern would hold up in their own deployment if they assumed the model would usually produce something parseable to correct in the first place.

Table 3: Stale or internally inconsistent figures identified by the manuscript-wide provenance audit (EXP11), disclosed rather than silently corrected or omitted. Row 1 is a live, unresolved data-integrity issue in the system’s own knowledge base, distinct from anything in this manuscript’s own text.

Description	Value(s)	Severity	Status
Live recall figure in the RAG knowledge base	0.695	HIGH	not fixed
Recall ambiguity: F1-argmax vs. configured point	0.719 vs. 0.782	MEDIUM-HIGH	disclosed here
Precision rounding: reported vs. raw re-derivation	0.858 vs. 0.859	LOW	footnoted
AQUA-7B agent-benchmark drift: published vs. re-run	60%/3.6 vs. 100%/2.0	MEDIUM	footnoted
EXP08 internal PSI resampling inconsistency ($\delta=0.04$)	0.7279 vs. 0.9070	LOW	footnoted
Vision leakage caveat not yet propagated to headline numbers	32 pairs, 14 adjacent	MEDIUM	resolved here (Table 2)

7.1 What a better-tuned model would and would not change

The most obvious objection to the central result is that AQUA-1B was tested under close to the worst plausible version of itself: an 8-example adapter its own training pipeline calls insufficient, with a properly sized alternative sitting unused. Whether the documented 128-example adapter would reduce the 50% unparseable rate is genuinely untested, and this paper does not claim to know the answer – it is stated here as open future work, not as a hedge against the current result. What is not open, and would not change regardless of which adapter is loaded, is the architecture’s independence from the model’s free-text reasoning: `apply_rule_override` never reads the `reasoning` field under either adapter, so a better-tuned model could raise the agreement rate without touching the property this paper actually measured. The two questions – does the model produce more parseable, more accurate output, and does the safety property hold regardless – are separable by design, and conflating them would understate how load-bearing the narrow-interface choice actually is.

A second, related question this system’s own agentic-routing result raises but cannot answer: whether the same escalation-only, narrow-interface discipline holds for agentic tool-routing at a larger on-device model scale. Both models tested here either failed outright (AQUA-1B) or produced a constant-answer artifact (AQUA-7B); neither result licenses a claim about where on the capability curve a model would need to sit before deterministic routing stops being the safer choice. That threshold, if it exists, was not found here, because it was not this paper’s question to answer – the negative result stands on its own terms, for these two models, on this scenario set.

7.2 The sparse-frame gap, stated as a gap

The reassuring empirical result that zero of 98 validation frames produced a full-frame miss is conditional on a property of the validation set that nobody designed in: every one of those 98 frames contains at least three ground-truth fish. Whether the same detector would miss a genuinely sparse frame – one or two fish, the case a welfare check most needs to catch, since an isolated or missing fish is itself a potential welfare signal – is a question this dataset cannot answer in either direction. The extrapolated 22–28% miss-risk estimate at $k = 1$ is exactly that: an extrapolation from an independence approximation, not a measurement, and it should not be read with the same confidence as the empirical 0/98 result it sits beside. Closing this gap requires collecting or synthesizing frames the current dataset simply does not contain, and that is stated here as a

concrete, actionable follow-up rather than a general caveat about validation-set size.

7.3 Practitioner rules this paper’s evidence actually supports

Four rules survive scrutiny against the specific evidence behind them, and are stated here at that scale rather than inflated into general design advice.

Architect a safety-critical LLM-plus-validator pipeline assuming the model will frequently produce nothing usable, verify that your fallback default is the conservative one, and never let your validator read anything from the model beyond the one enumerable field it is designed to certify. This is the paper’s central practitioner takeaway, and it follows directly from Section 6.1: the property that held was never “the model reasons well enough,” it was “the validator only ever looks at one field, and that field defaults safe.”

Tune a retrieval threshold for hallucination-averse domains against asymmetric cost, not raw F1. Section 6.4’s 0.85-over-0.84 choice sacrifices measurable recall specifically to drive hard-negative pass-through to zero, because a fabricated, confidently cited context actively misinforms while a missed document only weakens reasoning a downstream deterministic layer still adjudicates. The transferable lesson is the cost function a practitioner should write down before tuning, not this specific threshold value.

Choose a drift-detection binning scheme by sweeping your own confidence distribution’s severity range, not by copying a convention. Section 6.5’s crossover means “use quantile binning” is not a universal rule – it is a consequence of this particular detector’s confidence outputs being concentrated in a narrow band. A practitioner adopting PSI for a differently shaped output distribution should check for the same crossover before assuming either binning convention behaves as expected across their own severity range.

Benchmark the whole exported inference pipeline before attributing an accuracy loss to quantization. Section 6.3’s INT8-loses-less-than-fp32 finding means the standard mental model – lower numeric precision costs more accuracy – can be wrong for a given toolchain when the real loss sits in a shared post-processing path common to every export target. The actionable version of this rule is to measure the full configuration matrix, not to reason about precision loss in isolation.

A fifth pattern is worth naming even though it is not a design rule so much as a discipline: the adapter-provenance finding and the live 0.695 leak inside the system’s own RAG knowledge base (Section 6.6) are the same phenomenon at two different layers of the stack – an artifact silently drifted, or was mis-identified, between being produced and being used to support a conclusion. Both were caught by cheap, simple file-level checks (a record count and an mtime comparison; a repository-wide grep) that most teams skip because “we deployed the right thing” is treated as an assumption rather than a fact worth checking.

7.4 A hedged conjecture

These results suggest, but do not demonstrate beyond this one system, that narrow-enumerable-field validator interfaces may be a generally transferable safety-design principle for LLM-plus-validator pipelines outside aquaculture monitoring. The support for this is the consistency of the same underlying discipline across five architecturally distinct layers measured here – decision fusion, agentic tool routing, retrieval, MLOps retraining, and edge vision – not any test conducted outside this one system. This is offered as a direction worth testing in other domains, not as a demonstrated generalization, and a reader should weight it accordingly.

8 Known Limitations

This section states each limitation explicitly rather than letting any of them evaporate under space pressure. None is new information relative to Section 6; the point of collecting them here is that a reader assessing how far to trust this paper’s claims should be able to find every scope boundary in one place.

No live field deployment; this is an offline, bench-level evaluation. This is the most foundational scope boundary in the paper, stated first rather than buried among smaller caveats. No experiment reported in this paper was conducted with real physical sensors or inside an operating recirculating aquaculture facility – no physical water-quality sensor hardware exists for this system at all. Every measurement – the vision-model validation, the RAG retrieval tests, the AQUA-1B/rule-engine behavioral trace, the agentic tool-routing benchmark, the PSI drift-binning sweep, and the edge export benchmark – was run against synthetically generated sensor data, static labeled images, offline analysis scripts, and scripted or synthetic scenarios executed against the codebase, not against live sensor telemetry from an operating facility. This is a deliberate choice of study design, not an incidental gap: absent physical sensor hardware, the goal of this evaluation is to demonstrate the architectural feasibility of the proposed multi-component pipeline and to test whether its space and memory footprint is viable on realistic edge hardware, rather than to validate decision-making accuracy in a live setting; Section 1 and Section 5 state this framing directly. Only the system’s interface has ever been demonstrated, as a mockup/demo; the system itself has not been field-tested. Separately, and just as plainly: S.U.R.E. was built for the TEKNOFEST Agricultural Technologies competition, but this team did not enter that competition and did not pass its preliminary evaluation round. Every claim elsewhere in this paper about what the system does or how it performs should be read against this scope, not against an assumption of field deployment.

Sample size of the central result. The four-bucket dual-layer classification (Section 6.1) is measured on $n = 8$ – the evaluation harness’s entire fixed scenario population, not a sample drawn from a larger population. No confidence interval is implied by this count and none should be inferred from it. The finding is a census of a small, fixed scenario set, not a statistically powered estimate of a rate that would hold over a larger or different set of scenarios.

Single-device edge benchmarking. Every latency and export-format number in Section 6.3 was measured on a single Apple M4 Pro. The CoreML/ANE figure was re-measured across three independent process invocations to rule out single-session variance, but multi-session thermal-throttling effects beyond those three sessions, and behavior on any other device class, were not characterized and should not be assumed to transfer.

twin_bridge remains unresolved. The digital-twin comparison module is unit-tested only. It has never been exercised against a live Godot or CODESYS session, and no such session exists anywhere in this project’s history. This paper never describes it as field-validated, and a reader should not infer field validation from the passing unit-test count.

The adapter-provenance caveat is scoped narrowly. It applies to results derived from the dual-layer decision system (Section 6.1), which used the configured `sure-aqua-adapter`. It does not apply to the agentic tool-routing results (Section 6.2), which ran the base model with no adapter

loaded at all. Applying this caveat to the agentic-routing result would be a new factual error, not added precision, and this paper does not make that error anywhere in its text.

The vision dataset is real but small. Unlike the sensor values discussed above, the 510-image detector-training dataset (Section 4.2) is composed of genuine photographs captured from the project’s own physical camera setup and hand-labeled by the team, not synthetic imagery. It is, however, intentionally small at this stage, and the authors plan to expand it as future work; no claim in this paper should be read as asserting that 510 images is a sufficient long-run dataset size, only that it is what the reported detector was actually trained and validated on.

The validation set has no sparse-fish frames. Every one of the 98 validation frames used throughout Section 6.3 contains at least three ground-truth fish. The reassuring 0/98 full-frame-miss result is conditional on that property and says nothing about the $k = 1/k = 2$ regime, which this dataset does not contain at all. The 22–28% miss-risk figure sometimes cited alongside it is an extrapolation from an independence approximation, not a measurement, and should be read as one.

A live, unfixed data-integrity issue. The system’s RAG knowledge base (`llm-service/knowledge/06-davranis-ve-refah-gostergeleri.md`) still states the detector’s recall as approximately 0.695 as a live fact, a figure superseded elsewhere in this system’s history 43 minutes after it was introduced, but never corrected in this specific file. This file is ingested into the system’s vector store and is retrievable by the LLM running against it. It was not fixed during this audit, consistent with a read-only guardrail on the running system; it is disclosed here as a genuine, currently unresolved issue in the codebase, not a historical footnote.

Minor, non-conclusion-changing inconsistencies. A thousandths-place rounding difference exists between the commonly reported precision figure (0.858) and a freshly reproduced raw `val()` output (0.859); neither number changes any claim in this paper. AQUA-7B’s published agent-benchmark format-compliance and mean-step-count figures did not reproduce exactly on a fresh, code-unmodified re-run, with no configuration difference found to explain the drift, though the selection-accuracy and constant-answer conclusions reproduced exactly. The PSI drift-binning sweep’s own $\delta = 0.04$ entry differs from a separately drawn significant-drift window at the same nominal severity, an artifact of unseeded per-call resampling; no headline number in this paper depends on either individual value.

9 Conclusion

What this paper leaves a reader with is not primarily a new fact about AQUA-1B, but a claim about where a safety architecture’s soundness is allowed to live: never in the model’s reasoning, only in one enumerable field and a conservative default. Measured directly, half of the eight exercised scenarios produced no usable model output at all, and the rule engine acted alone; of the scenarios where it did not even need to act, because the model’s own final label already agreed, two of three reached that correct label through an invented sensor reading – a failure mode no evaluation that checks only the output label would ever see. None of this was measured against a model given a fair chance: the configured LoRA adapter is an 8-example smoke-test artifact the project’s own training pipeline calls insufficient, with a properly sized 128-example alternative sitting unused on disk throughout. That is the more informative condition to have tested under, not a caveat to apologize for – the safety property held while the model was given close to no chance to reason well, which pre-empts

the obvious reviewer objection that a better-tuned model would change the conclusion, rather than merely deferring it.

The same narrow-enumerable-interface discipline recurs across four other layers of this system, and it is not the same finding restated four times – each layer earns its place differently. The agentic tool-routing negative result survives an adversarial re-run designed to break it. The vision pipeline’s own honest self-correction (the configured operating point governs recall at 0.782, not the headline 0.719 the rest of this field would otherwise cite) makes the system’s safety picture more reassuring than a naive reading would suggest, precisely because it was measured rather than assumed. The RAG threshold and the PSI drift-binning gate are each careful, disclosed applications of an asymmetric-cost or crossover-aware design choice, not inventions. None of the five layers claims to have invented deterministic override – Safety Instrumented Systems and RL-safety shielding got there first, in different domains, for different kinds of probabilistic components – and this paper’s own contribution is narrower and, we think, more useful: a measured, implemented instance of that pattern applied to a local LLM in a new domain, plus a fabrication-detection finding that the enumerable-action-space literature this pattern descends from has no equivalent for, because it was never built to look inside a right decision for a wrong reason.

The practitioner rule this supports, stated in full in Section 7, is correspondingly general and falsifiable: assume the model will often say nothing usable, keep the fallback default conservative, and never let the validator see more of the model’s output than the one field it exists to certify. What would strengthen this claim further is already named rather than left implicit: a leakage-free retrain on a clip-disjoint re-split of the vision dataset, a live twin-bridge session against an actual Godot/CODESYS instance, and a test of the same narrow-interface discipline against a larger on-device model attempting agentic tool-routing. None of those would overturn what was measured here. They would tell us whether it holds under conditions this particular offline evaluation never happened to create.

Acknowledgements

S.U.R.E. was built for the TEKNOFEST Agricultural Technologies competition by Batuhan Çıtak and Erdem Sabri Veli. We thank the competition organizers for a problem framing concrete enough to motivate an implemented, bench-evaluated system rather than a purely theoretical one.

We partially used pAI/MSc for this manuscript [Abdelmoneum et al., 2026].

A Supplementary Tables

This appendix carries derivation-style detail that would otherwise bloat Section 6: the full per-scenario record behind the four-bucket classification, and the full 16-point PSI severity sweep summarized in Figure 5.

A.1 EXP03: full per-scenario record

Tally across all 8 reasoning strings: 1 accurate, 1 uncodable (empty generation), and 6 inaccurate in some respect against the true sensor snapshot – of which exactly 3 (T02, T05, T08, bolded above) fabricate a specific numeric sensor value that was never in the input; the other 3 (T03, T06, T07) are inaccurate without inventing a number (an unsupported blanket claim, or an internally incoherent reading of a plausibly real value). Note T07: even though the raw model output was unparseable, the safe default (ok) was still correctly escalated to **critical** by `apply_rule_override`, because the

Table 4: All 8 evaluation-harness scenarios: rule-engine status, AQUA-1B’s pre-override status, the post-override final status, the resulting bucket, and the manual reasoning-accuracy coding.

ID	Scenario	Rule	Pre	Post	Bucket	Reasoning coding
T01	Normal conditions	ok	ok	ok	agrees	accurate
T02	Critical DO (5.7 mg/L)	critical	ok	critical	under-calls, escalated	fabricated (6.0 vs. 5.7)
T03	pH warning	warning	ok (default)	warning	unparseable	inaccurate, not fabricated
T04	Optimal conditions	ok	ok (default)	ok	unparseable	uncodable (empty)
T05	Cold water + low pH	warning	warning	warning	agrees	fabricated , right label
T06	High O2, high activity	ok	ok (default)	ok	unparseable	inaccurate, not fabricated
T07	Multi-parameter emergency	critical	ok (default)	critical	unparseable	inaccurate, severe, not fabricated
T08	No fish detected	warning	warning	warning	agrees	fabricated , right label

override compares the *defaulted* status against the rule engine’s, not the raw text – the escalate-only safety net functions even when the model’s output is garbage.

A.2 EXP08: full 16-point PSI severity sweep

Table 5: Full mean-shift severity sweep ($n = 1467$ reference/current window), quantile (decile) vs. equal-width PSI, underlying Figure 5.

δ	Quantile PSI	Equal-width PSI
0.00	0.0050	0.0049
0.02	0.1639	0.0776
0.04	0.7279	0.3059
0.06	1.4778	0.9057
0.08	2.1842	2.1412
0.10	2.6520	3.0270
0.12	3.3716	3.4738
0.14	3.4993	3.8634
0.16	3.9860	4.2552
0.18	4.0302	5.1331
0.20	4.1238	5.9264
0.22	4.3192	6.2131
0.24	4.6096	6.3534
0.26	4.8597	6.7538
0.28	4.8228	7.3928
0.30	4.8444	7.6493

A.3 EXP04: full 9-scenario agentic tool-routing record

AQUA-7B’s rise from 50% to 71.4% raw selection accuracy on the expanded set is not evidence of improved discrimination – it follows directly from three of the four new scenarios being sensor-trend-relevant by design, so a model emitting a single constant answer scores well on any test whose

Table 6: Aggregate agentic tool-routing results, original 5-scenario set vs. the expanded 9-scenario set (5 original + 4 newly authored, adversarial-by-design scenarios).

Model	Set	Format %	Sel. %	Steps	Constant answer	
AQUA-1B	$n=5$	0%	0%	0.0	false	
AQUA-1B	$n=9$	0%	0%	0.0	false	
AQUA-7B	$n=5$	100%	50%	2.0	true	(5/5)
					<code>get_sensor_trend</code>	
AQUA-7B	$n=9$	100%	71.4%	2.0	true	(9/9)
					<code>get_sensor_trend</code>	
AQUA-7B (published README, historical)	$n=5$	60%	50%	3.6	true (5/5)	

answer key frequently contains that answer. The constant-answer flag, confirmed true on both sets, is the metric this paper treats as load-bearing; the raw selection percentage is not.

References

- Mahmoud Abdelmoneum, Pierfrancesco Beneventano, and Tomaso Poggio. pai/msc: ML theory research with humans on the loop. Technical Report Technical Report v0, MIT, 2026. URL <https://dspace.mit.edu/handle/1721.1/165377>.
- AgMRC. Water quality management for recirculating aquaculture. AgMRC, 2026. URL https://www.agmrc.org/media/cms/FA3A_561BD17CD3C4D.pdf. Accessed 2026-08-26.
- Gopichand Agnihotram and Joydeep Sarkar. Evaluating precision and recall at retrieval time in retrieval-augmented generation (rag) systems. ResearchGate, 2025. URL https://www.researchgate.net/publication/397055775_Evaluating_Precision_and_Recall_at_Retrieval_Time_in_Retrieval-Augmented_Generation_RAG_Systems. Accessed 2026-08-26.
- AICHE. What is a safety instrumented system? CCPS Beacon, 2009. URL <https://www.aiche.org/sites/default/files/beacon-article/2009-07-Beacon-English.pdf>. Accessed 2026-08-26.
- Anonymous. Autodrift: An end-to-end mlops pipeline with automated data drift detection and triggered model retraining. ResearchGate; author identity and peer-review status unconfirmed despite search, 2025. URL https://www.researchgate.net/publication/404272325_AutoDrift_An_End-to-End_MLOps_Pipeline_with_Automated_Data_Drift_Detection_and_Triggered_Model_Retraining. Accessed 2026-08-26.
- Arize AI. Data binning challenges in production: How to bin to win. Arize AI Blog, 2026a. URL <https://arize.com/blog-course/data-binning-production/>. Accessed 2026-08-26.
- Arize AI. Population stability index (psi): What you need to know. Arize AI Blog, 2026b. URL <https://arize.com/blog/population-stability-index-psi/>. Accessed 2026-08-26.
- Arthur.ai. Ai agent guardrails: Pre-llm & post-llm best practices. Arthur.ai Blog, 2025. URL <https://www.arthur.ai/blog/best-practices-for-building-agents-guardrails>. Accessed 2026-08-26.

- Nishant Balepur, Abhilasha Ravichander, and Rachel Rudinger. Artifacts or abduction: How do llms answer multiple-choice questions without the question? arXiv:2402.12483, 2024. URL <https://arxiv.org/pdf/2402.12483>. Accessed 2026-08-26.
- Sanjay Basu and Benjamin Huynh. Mitigating hallucinations in healthcare ai: a systematic review of evidence-based strategies. *BMC Health Services Research*, 2026. URL <https://link.springer.com/article/10.1186/s12913-026-14851-1>.
- Braintrust. Rag evaluation metrics: How to evaluate your rag pipeline with braintrust. Braintrust Articles, 2025. URL <https://www.braintrust.dev/articles/rag-evaluation-metrics>. Accessed 2026-08-26.
- Heather Browning. Improving welfare assessment in aquaculture. *Frontiers in Veterinary Science*, 2023. URL <https://www.frontiersin.org/journals/veterinary-science/articles/10.3389/fvets.2023.1060720/full>.
- Davide Corsi, Guy Amir, Andoni Rodriguez, Cesar Sanchez, Guy Katz, and Roy Fox. Verification-guided shielding for deep reinforcement learning. arXiv:2406.06507, 2024. URL <https://arxiv.org/pdf/2406.06507>. Accessed 2026-08-26.
- Y. Cui et al. Fish tracking, counting, and behaviour analysis in digital aquaculture: A comprehensive survey. *Reviews in Aquaculture*, 2025. URL <https://onlinelibrary.wiley.com/doi/10.1111/raq.13001>. Preprint: arXiv:2406.17800.
- Anna Danilec. Deterministic guardrails for non-deterministic agents. dev.to, 2026. URL https://dev.to/anna_danilec/deterministic-guardrails-for-non-deterministic-agents-127b. Accessed 2026-08-26.
- DataRobot. Introducing mlops champion/challenger models. DataRobot Blog, 2026. URL <https://www.datarobot.com/blog/introducing-mlops-champion-challenger-models/>. Accessed 2026-08-26.
- Angelo Ferrando. Watchdogs and oracles: Runtime verification meets large language models for autonomous systems. arXiv:2511.14435, 2025. URL <https://arxiv.org/pdf/2511.14435>. Accessed 2026-08-26.
- Fiddler AI. Measuring data drift with the population stability index (psi). Fiddler AI Blog, 2026. URL <https://www.fiddler.ai/blog/measuring-data-drift-population-stability-index>. Accessed 2026-08-26.
- D. Fitzgerald et al. Machine vision applications for welfare monitoring in aquaculture: Challenges and opportunities. *Aquaculture, Fish and Fisheries*, 2025. URL <https://onlinelibrary.wiley.com/doi/full/10.1002/aff2.70036>.
- Saumya Goswami and Siddharth Kurra. Halt-rag: A task-adaptable framework for hallucination detection with calibrated nli ensembles and abstention. arXiv:2509.07475, 2025. URL <https://arxiv.org/pdf/2509.07475>. Accessed 2026-08-26.
- Kait Healy, Bharathi Srinivasan, Visakh Madathil, and Jing Wu. Internal representations as indicators of hallucinations in agent tool selection. arXiv:2601.05214, 2026. URL <https://arxiv.org/abs/2601.05214>. Accessed 2026-08-26.

- Wentao Huang, Yangfeng Wang, Jie Xia, Xinyi Jin, Hongliang Zhu, Branko Glamuzina, Wenying Yu, and Xiaoshuan Zhang. Non-destructive classification of sturgeon stress using cross-modal data fusion and multi-input deep learning models. *Computers and Electronics in Agriculture*, 2024. URL <https://www.sciencedirect.com/science/article/abs/pii/S0168169924002941>.
- IChemE. Applying the latest standard for functional safety — iec 61511. IChemE, 2026. URL <https://www.icheme.org/media/9906/xviii-paper-23.pdf>. Accessed 2026-08-26.
- Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Ye Jin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):Article 248, 2023. doi: 10.1145/3571730. URL <https://dl.acm.org/doi/10.1145/3571730>. Also arXiv:2202.03629. Accessed 2026-08-26.
- Toghrul Karimov, Hassan Imani, and Allan Kazakov. Quantization robustness to input degradations for object detection. arXiv:2508.19600, 2025. URL <https://arxiv.org/pdf/2508.19600>. Accessed 2026-08-26.
- Bettina Könighofer, Roderick Bloem, Nils Jansen, Sebastian Junges, and Stefan Pranger. Shields for safe reinforcement learning. *Communications of the ACM*, 2025. URL <https://dl.acm.org/doi/10.1145/3715958>.
- Xixun Lin et al. Llm-based agents suffer from hallucinations: A survey of taxonomy, methods, and directions. arXiv:2509.18970, 2025. URL <https://arxiv.org/html/2509.18970v1>. Accessed 2026-08-26.
- P. Lindholm-Lehto. Water quality monitoring in recirculating aquaculture systems. *Aquaculture, Fish and Fisheries*, 2023. URL <https://onlinelibrary.wiley.com/doi/10.1002/aff2.102>.
- Omar, Sorin, Collins, Reich, Freeman, Gavin, Charney, Stump, Bragazzi, Nadkarni, and Klang. Multi-model assurance analysis showing large language models are highly vulnerable to adversarial hallucination attacks during clinical decision support. *Communications Medicine*, 2025. URL <https://www.nature.com/articles/s43856-025-01021-3>.
- ONNX community. Inference time discrepancy when using torchscript vs onnx exported model. GitHub Issue #6287, onnx/onnx and #21689, microsoft/onnxruntime, 2024. URL <https://github.com/onnx/onnx/issues/6287>. Accessed 2026-08-26.
- Roboflow. Comparing cloud and on-device inference for computer vision. Roboflow Blog, 2025. URL <https://blog.roboflow.com/comparing-cloud-and-on-device-inference/>. Accessed 2026-08-26.
- Rulebricks. Deterministic guardrails for llms: Building safe, auditable ai systems. Rulebricks Blog, 2026. URL <https://rulebricks.com/blog/deterministic-guardrails-for-llms-building-safe-auditable-ai-systems>. Accessed 2026-08-26.
- Raghav Sharma and Manan Mehta. Small language models for agentic systems: A survey of architectures, capabilities, and deployment trade-offs. arXiv:2510.03847, 2025. URL <https://arxiv.org/pdf/2510.03847>. Accessed 2026-08-26.
- Debu Sinha. The semantic illusion: Certified limits of embedding-based hallucination detection in rag systems. arXiv:2512.15068, 2026. URL <https://arxiv.org/pdf/2512.15068>. Accessed 2026-08-26.

- Snowflake. Automated model retraining & deployment in snowflake. Snowflake Developer Guides, 2026. URL <https://www.snowflake.com/en/developers/guides/ml-champion-challenger-model-deployment/>. Accessed 2026-08-26.
- Yang Tian, Zhengpeng Shi, Yu Zhou, and Bo Zhao. Beyond function calling: Benchmarking tool-using agents under tool-environment unreliability. arXiv:2606.25819, 2025. URL <https://arxiv.org/html/2606.25819>. Accessed 2026-08-26.
- Linda Tschirren, David Bachmann, Ali Cem Güler, Oliver Blaser, Nicola Rhyner, Andreas Seitz, Erich Zbinden, Thomas Wahli, Helmut Segner, and Dominik Refardt. Myfishcheck: A model to assess fish welfare in aquaculture. PMC7826897, 2021. URL <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC7826897/>. Accessed 2026-08-26.
- Ultralytics. Coreml export for yolo models. Ultralytics Docs, 2026. URL <https://docs.ultralytics.com/integrations/coreml>. Accessed 2026-08-26.
- Ultralytics Academy. Optimize with tensorrt. YOLO in Production course, 2025. URL <https://academy.ultralytics.com/courses/yolo-in-production/optimize-with-tensorrt>. Accessed 2026-08-26.
- Ultralytics community. Poor results when detecting object with exported model (onnx, torchscript). GitHub Issue #8772, ultralytics/yolov5, 2022. URL <https://github.com/ultralytics/yolov5/issues/8772>. Accessed 2026-08-26.
- Javal Vyas and Mehmet Mercangoz. Autonomous control leveraging llms: An agentic framework for next-generation industrial automation. arXiv:2507.07115, 2025. URL <https://arxiv.org/html/2507.07115>. Accessed 2026-08-26.
- Liang Wang et al. Text embeddings by weakly-supervised contrastive pre-training. arXiv:2212.03533, 2022. URL <https://arxiv.org/pdf/2212.03533>. Accessed 2026-08-26.
- Anak Wannaphaschaiyong. Why small llms fail at tool calling: The shocking discovery from our llama 3b benchmark. dev.to, 2026. URL https://dev.to/anak_wannaphaschaiyong_11/why-small-llms-fail-at-tool-calling-the-shocking-discovery-from-our-llama-3b-benchmark-5lg. Accessed 2026-08-26.
- WWT. Mlops and drift: Reducing risk and ensuring robust ml models. WWT Article, 2026. URL <https://www.wwt.com/article/mlops-and-drift-reducing-risk-and-ensuring-robust-ml-models>. Accessed 2026-08-26.
- Yuchen Xia, Nasser Jazdi, Jize Zhang, Chaitanya Shah, and Michael Weyrich. Control industrial automation system with large language model agents. arXiv:2409.18009, 2024. URL <https://arxiv.org/abs/2409.18009>. Accessed 2026-08-26.
- Bilal Yurdakul. *Statistical Properties of Population Stability Index*. PhD thesis, Western Michigan University, 2018. URL <https://scholarworks.wmich.edu/dissertations/3208/>. ScholarWorks at WMU, Dissertation 3208. Accessed 2026-08-26.
- Yifu Zhang et al. Bytetrack: Multi-object tracking by associating every detection box. In *European Conference on Computer Vision (ECCV)*, 2022. URL <https://arxiv.org/abs/2110.06864>. arXiv:2110.06864.